

Session: Generative AI. Autoencoders and GANs

Javier Serrano

Applied Machine Learning
Master in Data Science and Analytics



Strathmore University

@iLabAfrica Centre

Representation and Generative Learning

AUTOENCODERS

- Latent representations or codings
- Data unlabelled (no supervision)
- Useful for dimensionality reduction
- Useful for feature extraction in unsupervised pretraining
- Generative models

GENERATIVE ADVERSARIAL NETWORKS (GANs)

- Generating realistic AI Images
- Increasing resolution of images, colorization
- Generating other types of data

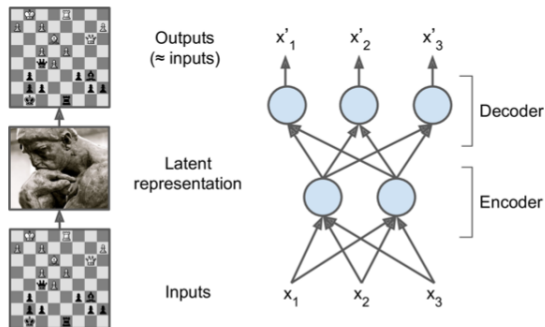
Summary

Autoencoders

Generative Adversarial Networks

Efficient Data Representation

1. Autoencoder looks at the inputs, converts them to an efficient **latent representation**, and then spits out something close to the inputs
2. An autoencoder is composed of two parts: an encoder that converts the inputs to a latent representation, followed by a decoder that converts the internal representation to the outputs



Efficient Data Representation

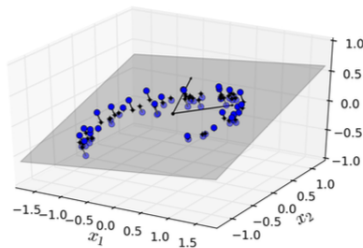
Example: Dimensionality Reduction - PCA

```
from tensorflow import keras
```

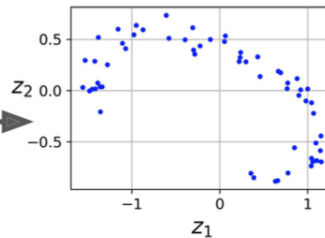
```
encoder = keras.models.Sequential([keras.layers.Dense(2, input_shape=[3])])  
decoder = keras.models.Sequential([keras.layers.Dense(3, input_shape=[2])])  
autoencoder = keras.models.Sequential([encoder, decoder])
```

```
autoencoder.compile(loss="mse", optimizer=keras.optimizers.SGD(lr=0.1))
```

Original 3D dataset

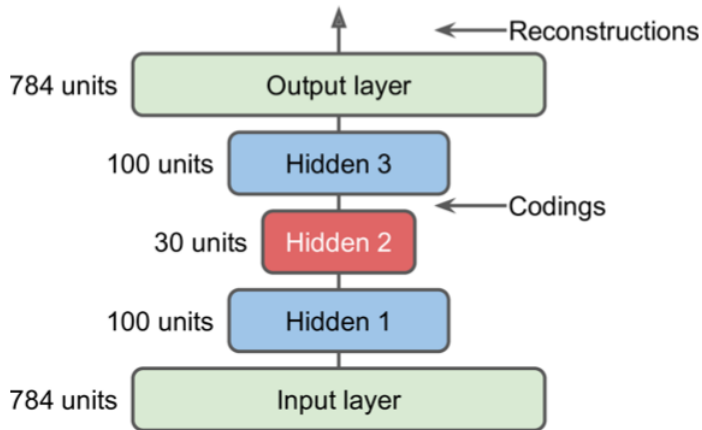


2D projection with max variance



Efficient Data Representation

Stacked/Deep Autoencoders



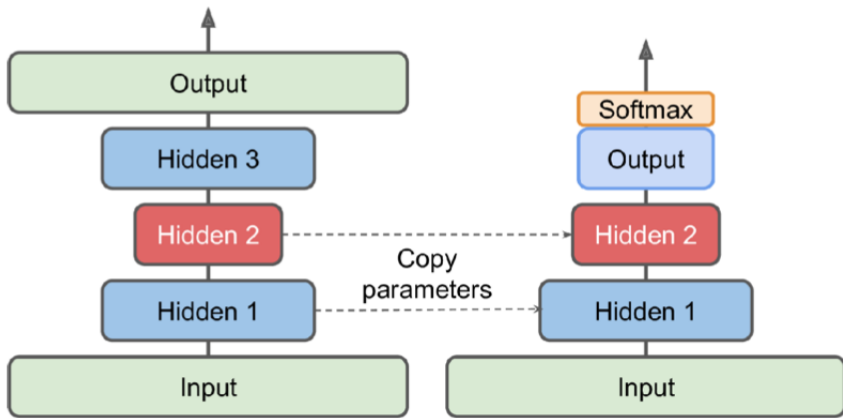
Stacked/Deep Autoencoders

Implementation in Keras

```
stacked_encoder = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(100, activation="selu"),
    keras.layers.Dense(30, activation="selu"),
])
stacked_decoder = keras.models.Sequential([
    keras.layers.Dense(100, activation="selu", input_shape=[30]),
    keras.layers.Dense(28 * 28, activation="sigmoid"),
    keras.layers.Reshape([28, 28])
])
stacked_ae = keras.models.Sequential([stacked_encoder, stacked_decoder])
stacked_ae.compile(loss="binary_crossentropy",
                   optimizer=keras.optimizers.SGD(lr=1.5))
history = stacked_ae.fit(X_train, X_train, epochs=10,
                        validation_data=[X_valid, X_valid])
```

Stacked/Deep Autoencoders

Unsupervised Pretrained



Phase 1

Train the autoencoder
using all the data

Phase 2

Train the classifier
on the labeled data

Other kind of Autoencoders

Convolutional and Recurrent Autoencoders

Convolutional Autoencoders .

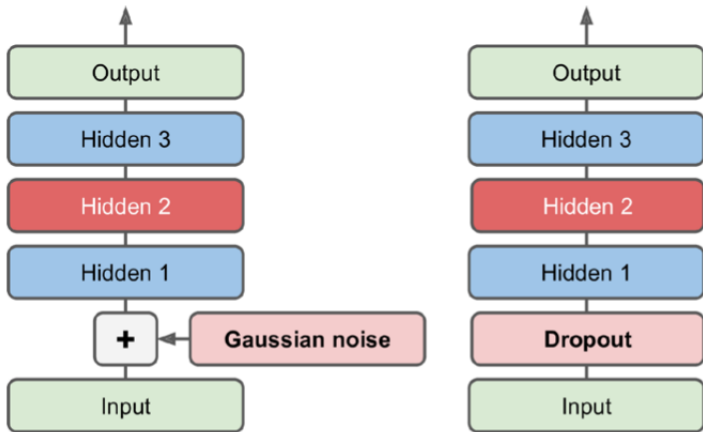
- The encoder is a regular CNN composed of convolutional layers and pooling layers. Reduces the spatial dimensionality of the inputs while increasing the depth (feature maps).
- The decoder must do the reverse. You can use transpose convolutional layers.

Recurrent Autoencoder .

- The encoder is typically a sequence-to-vector RNN which compresses the input sequence down to a single vector.
- The decoder is a vector-to-sequence RNN that does the reverse.

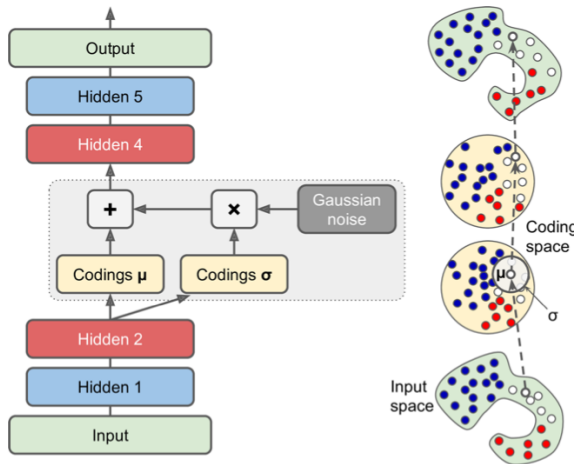
Denoising Autoencoders

A way to force the autoencoder to learn useful features, and/or remove noise from images



Variational Autoencoder

- Probabilistic autoencoders, their outputs are partly determined by chance, even after training
- Generative autoencoder



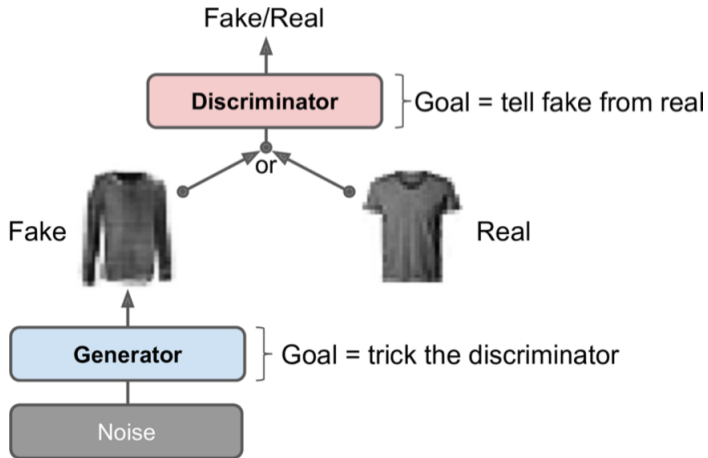
Summary

Autoencoders

Generative Adversarial Networks

Generative Adversarial Networks

Ian Goodfellow et al. 2014



Generative Adversarial Networks

How to train this networks?

1. Train the Discriminator.

- Use equal number of fake (0) and real (1) images.
- It is a binary classifier and use cross-entropy loss.
- We use backpropagation optimazing only discrیمانator weights

2. Train the Generator.

- Produce a batch of fake images
- Use the discriminator to classify these images
- Set the labels to 1 to force the generator to produce images as real as possible.
- The weights of the discriminator are frozen during this step, backpropagation only affects the weights of the generator.

Generative Adversarial Networks

The Difficulties of Training GANs

- **Mode collapse.** The generator's outputs gradually become less diverse.
 - The generator gets better at producing convincing one sort of images
 - The discriminator encourage to produce the same sort of images
 - At some point the discriminator will discriminate a fake image, and the generator will be force to move to another class.
 - Because the generator and the discriminator are constantly pushing against each other, their parameters may end up oscillating and becoming unstable
- **Solutions.**
 - *Experience replay.* Avoid overfit the latest generator's outputs.
 - *Mini-batch discrimination.* To keep diversity

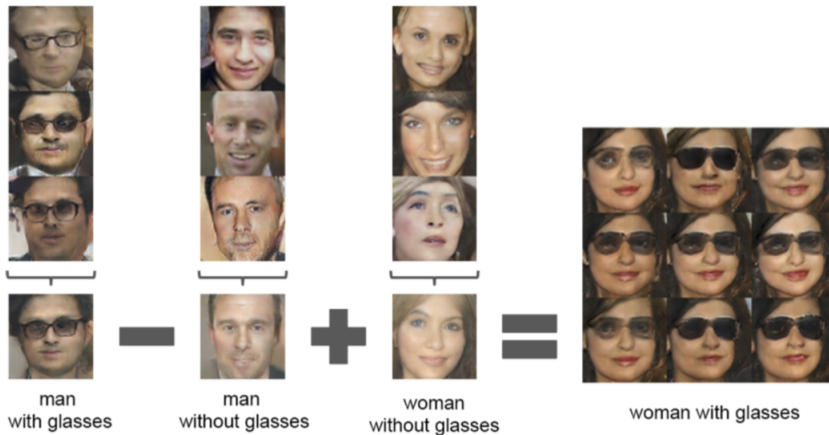
Generative Adversarial Networks

Deep Convolutional GANs (DCGAN)

- Replace any pooling layers with strided convolutions (in the discriminator) and transposed convolutions (in the generator).
- Use Batch Normalization in both the generator and the discriminator, except in the generator's output layer and the discriminator's input layer.
- Remove fully connected hidden layers for deeper architectures.
- Use ReLU activation in the generator for all layers except the output layer, which should use tanh.
- Use leaky ReLU activation in the discriminator for all layers.

Generative Adversarial Networks

DCGAN: Vector arithmetic for visual concepts



References

- A. Geron, *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, 2019.
- Andrew Ng course on Deep Learning
<https://www.youtube.com/watch?v=E13qqHb3J7U>
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.
<https://github.com/janishar/mit-deep-learning-book-pdf>
- Simon J.D. Prince *Understanding Deep Learning* 2024
<https://udlbook.github.io/udlbook/>

Thank you